

MICROPROCESSOR AND MICROCONTROLLER

2 MARKS Q&A

Y2/S4

UNIT V

Microcontroller: Intel 8051 Microcontroller: Introduction – Architecture – Memory Organization – Special Function Registers – Pins and Signals – Timing and control – Port Operation – Memory and I/O interfacing – Interrupts – Instruction Set and Programming.

1. Differentiate Microprocessor and Microcontroller.***

Microcontroller	Microprocessor
* Microcontroller having inbuilt RAM or ROM and inbuilt Timer * It includes MPU, RAM, ROM and I/O lines. * It is a single chip micro computer. * Input and Outputs ports are available. * Microcontroller has inbuilt serial port. * Separate memory to store program and data. * It has many function on IC * Boolean operation directly possible. * It takes few instructions to read and write data from external memory.	*Microprocessor do not have inbuilt RAM or ROM and timer. * It is used as CPU. * It is a programmable integrated device that has computing and decision making capability. *Input and Output ports are not available. *It do not have inbuilt serial port, requires 8250 device. * Program and data are stored in the same memory. * It has less multifunction pins on IC. *Boolean operation is not possible directly. * It takes many instructions to read and write data from external memory.

2. Write about Interrupt in 8051.***

The 8051 has **five** interrupt sources. Each of these interrupts can be programmed to two priority levels. The interrupts sources are:

Source	Description
INT0	External request from P3.2 pin.
Timer 0	Overflow from Timer 0 activates the Interrupt Request Flag TF0.
INT1	External request from P3.3 pin.
Timer 1	Overflow from Timer 1 activates the interrupt Request Flag TF1.
Serial port	Completion of transmission or reception of a serial frame activates the flags TI or RI.

Source	Priority level
External interrupt 0	Highest
Timer 0 overflow	↓
External interrupt 1	↓
Timer 1 overflow	↓
Serial port	lowest

MICROPROCESSOR AND MICROCONTROLLER

2 MARKS Q&A

Y2/S4

3. List some special function registers in 8051.***

- All the resources in the 8051 can be accessed through special function register maintained in the internal data memory.
- There are **21** special function registers in the 8051.
- Some special function register are bit addressable.
- The various special function registers are:

ACC	-	Accumulator*
B	-	B Register*
PSW	-	Program Status Word*
SP	-	Stack Pointer
DPTR (Low)	-	Data Pointer Low
DPTR (High)	-	Data Pointer High
P0	-	Port 0*
P1	-	Port 1*
P2	-	Port 2*
P3	-	Port 3*
IP	-	Interrupt Priority*
IE	-	Interrupt Enable*
TMOD	-	Timer/Counter Mode
TCON	-	Timer/Counter control*
TH0	-	(Timer/counter) 0 High
TL0	-	(Timer/Counter) 0 Low
TH1	-	(Timer/Counter) 1 High
TH2	-	(Timer/Counter) 1 Low
SCON	-	Serial Control*
SBUF	-	Serial Data Buffer
PCON	-	Power Control

* These bytes are bit addressable as well.

4. Define Microcontroller.***

Microcontroller is a single chip that contains the processor (CPU), non-volatile memory for the program (ROM), volatile memory for input and output (RAM), a clock and an I/O control unit. Also called as “COMPUTER ON A CHIP”.

Microcontrollers are in all sizes and architectures, with the smaller, commodity chips costing as little as 50 cents in quantities of 10,000.

5. What are the flags in PSW?***

The Program Status Word (PSW) is one of the Special Function Registers and contains the carry bit, the auxiliary carry bit (for BCD operations), two register select bits, the overflow flag bit, a parity bit and two user definable status flag bits.

MICROPROCESSOR AND MICROCONTROLLER

2 MARKS Q&A

Y2/S4

The format of Program Status Word

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
CY	AC	F0	RS1	RS0	OV	-	P

User Flag 0

Reserved

CY – Carry Flag is affected after the execution of an arithmetic and logic operation.

Carry = 1 (set)

No carry = 0 (cleared)

The carry flag is used to detect the errors in unsigned arithmetic operations.

AC – Auxiliary Carry is set, if the operation results in carry out of low-order 4 bits of the result (during addition) or a borrow from the high-order bits into low-order 4 bits (during subtraction); otherwise AC is cleared. This is useful to perform BCD operations.

F0 – Flag 0, Available to the user for general purpose.

RS0, RS1 – Register bank select control bits 0 & 1 and represent the current register bank in the Internal Data RAM selected.

RS1 RS0

0 0 Bank 0 (00_h to 07_h)

0 1 Bank 1 (08_h to 0F_h)

1 0 Bank 2 (10_h to 17_h)

1 1 Bank 3 (18_h to 1F_h)

OV – Overflow Flag is set, if the operation results in a carry into the high-order bit of the result but not the carry out of the high-order bit, or vice versa; otherwise OV is cleared. The overflow flag is only used to detect errors in signed arithmetic operations.

P- Parity flag is useful for serial communication. This flag reflects the number of 1s in the accumulator only. If the accumulator contains an odd number of 1s, then bit is set to 1. If the accumulator contains an even number of 1s, then it is cleared.

6. What are the addressing modes of Intel 8051?***

The instructions of 8051 are classified into **five** modes based on the source and destination type.

1. Register addressing mode.
2. Direct addressing mode.
3. Register-indirect addressing mode
4. Immediate addressing mode.
5. Base + Offset addressing mode (Base register plus index register indirect).

7. What are the advantages of using Microcontroller over Microprocessor?***

Advantages of using Microcontroller are:

- ❖ Reduced size and complexity.
- ❖ Fastest access to its devices.
- ❖ Lower power consumption.

MICROPROCESSOR AND MICROCONTROLLER

2 MARKS Q&A

Y2/S4

- ❖ User friendly
- ❖ Many programs are inbuilt s library functions.
- ❖ Application oriented.
- ❖ Capable of executing programs dedicatedly without much user intervention.
- ❖ Achieves portability easily.
- ❖ Best suitable for embedded systems.

8. What are the features of Microcontroller 8051?

The features of Microcontroller 8051 family are

- 8 bit CPU
- On-Chip oscillator and clock circuit
- 32 Input / Output lines.
- Capable of accessing 64K of external program memory
- Capable of accessing 64K of external data memory
- Two 16-bit Timer/Counter
- Two external Interrupts with Priority levels
- Boolean processor
- Full duplex serial port

9. Discuss about the port operation.***

Port 0-3: There are 4 bidirectional input/output ports of 8 bits each. These are directly represented as pins of the 8051 chips and are bit addressable. These are multifunctional resources that can be programmed based on the application needs, for example

- **P0** and **P2** can be used as I/O ports or address lines for external memory.

Port 0 provides multiplexing of address and data. It consists of P0.0-P0.7 and also AD₀ to AD₇ (Lower order address bits).

Port 2 consists of P2.0 to P2.7 and provides the high order address byte (AD₈ to AD₁₅) during access to external memory.

• **Port 1** consists of P1.0 to P1.7. This port is meant for user's purpose and also used for the low order address byte during programming and verification. It plays an important role in programming of internal memory of the 8751 and 8051.

• **Port 3** It consists of P3.0 to P3.7 and can be used as I/O port. It also serves the function of various special features of the 8051 family as given below.

Port	Alternate Function
P3.0	RxD Serial input port.
P3.1	TxD Serial output port
P3.2	INT0 External interrupt
P3.3	INT1 External interrupt
P3.4	T0 Timer 0 External input

MICROPROCESSOR AND MICROCONTROLLER

2 MARKS Q&A

Y2/S4

P3.5	T1	Timer 1 External input
P3.6	WR	Write strobe for external memory
P3.7	RD	Read strobe for external memory

10. What are the functions of $\overline{\text{PSEN}}$ pin?***

Program Store Enable ($\overline{\text{PSEN}}$): This pin gives out active low output pulses and used for fetching the data from the external program memory.

When the device is executing out of external memory, $\overline{\text{PSEN}}$ is activated and $\overline{\text{PSEN}}$ is not activated when the device is executing out of internal memory.

➤ $\overline{\text{PSEN}}$ is the control signal used to enable the external program memory.

11. What is the function of EA pin?***

EA/ V_{pp} : This pin, if connected to 5 V, will indicate the processor that the 4KB of the program memory within the chip should be used. Hence, address 0000_H to 0FFF_H will be within the chip. Once the address exceeds this, i.e. 1000_H to FFFF_H, the external program memory is accessed.

External Address (EA): When it is held HIGH, the CPU executes the instructions from the internal program ROM memory (unless the program counter exceeds 0FFF_H in 8051 or 1FFF_H in 8052). Holding EA pin LOW forces the CPU to execute the instructions from the external memory regardless of the program counter value. In 8751, this pin also receives the 21V programming supply voltage (V_{pp}) during EPROM programming.

12. What is Microcontroller and Microcomputer

Microcontroller is a device that includes microprocessor; memory and I/O signal lines on a single chip, fabricated using VLSI technology. Microcomputer is a computer that is designed using microprocessor as its CPU. It includes microprocessor, memory and I/O.

13. Explain DJNZ instructions of intel 8051 microcontroller?***

- DJNZ Rn, rel Decrement the content of the register Rn and jump if not zero.
- DJNZ direct, rel Decrement the content of direct 8-bit address and jump if not zero

14. Explain the 16 bit registers DPTR and SP of 8051.**

DPTR:

DPTR stands for data pointer. DPTR consists of a high byte (DPH) and a low byte (DPL). Its function is to hold a 16-bit address. It may be manipulated as a 16-bit data register or as two

MICROPROCESSOR AND MICROCONTROLLER

2 MARKS Q&A

Y2/S4

independent 8-bit registers. It serves as a base register in indirect jumps, lookup table instructions and external data transfer.

SP:

SP stands for stack pointer. SP is a 8- bit wide register. It is incremented before data is stored during PUSH and CALL instructions. The stack array can reside anywhere in on-chip RAM. The stack pointer is initialised to 07H after a reset. This causes the stack to begin at location 08H.

15. Write a program to load accumulator A ,DPH and DPL with 30H.

```
MOV A,#30
```

```
MOV DPH,A
```

```
MOV DPL,A
```

Unit V

Microcontroller: Intel 8051 Microcontroller: Introduction – Architecture – Memory Organization – Special Function Registers – Pins and Signals – Timing and control – Port Operation – Memory and I/O interfacing – Interrupts – Instruction Set and Programming.

1. INTRODUCTION

A **microcontroller** (sometimes abbreviated **μC**, **uC** or **MCU**) is a small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals. Program memory in the form of NOR flash or OTP ROM is also often included on chip, as well as a typically small amount of RAM. Microcontrollers are designed for embedded applications, in contrast to the microprocessors used in personal computers or other general purpose applications.

Microcontrollers are used in automatically controlled products and devices, such as automobile engine control systems, implantable medical devices, remote controls, office machines, appliances, power tools, toys and other embedded systems. By reducing the size and cost compared to a design that uses a separate microprocessor, memory, and input/output devices, microcontrollers make it economical to digitally control even more devices and processes. Mixed signal microcontrollers are common, integrating analog components needed to control non-digital electronic systems.

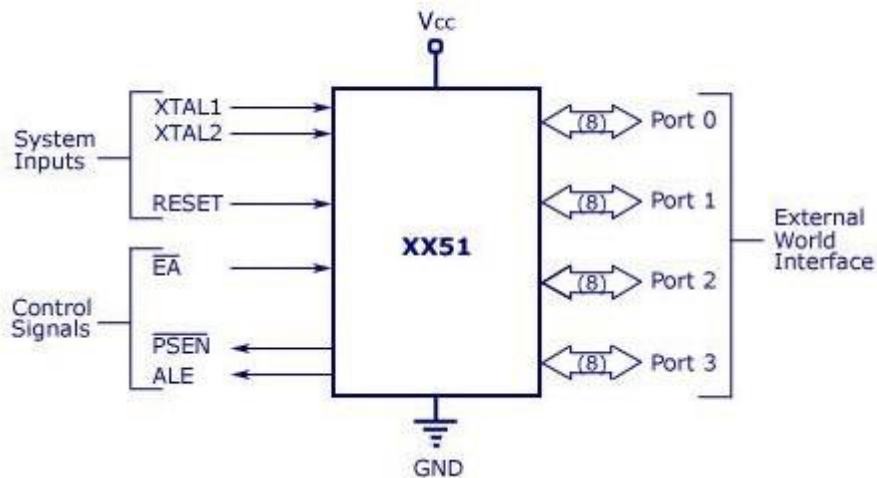
Some microcontrollers may use four-bit words and operate at clock rate frequencies as low as 4 kHz, for low power consumption (single-digit milliwatts or microwatts). They will generally have the ability to retain functionality while waiting for an event such as a button press or other interrupt; power consumption while sleeping (CPU clock and most peripherals off) may be just nanowatts, making many of them well suited for long lasting battery applications. Other microcontrollers may serve performance-critical roles, where they may need to act more like a digital signal processor (DSP), with higher clock speeds and power consumption.

2. 8051 MICROCONTROLLER ARCHITECTURE

It's possible to explain microcontroller architecture to a great detail, but we are limiting scope of this article to internal architecture, pin configuration, program memory and data memory organization. The basic architecture remains same for the MCS-51 family. In general all microcontrollers in MCS- 51 family are represented by XX51, where XX can take values like 80, 89 etc.

Schematic and Features

XX51 schematic Inputs and Outputs

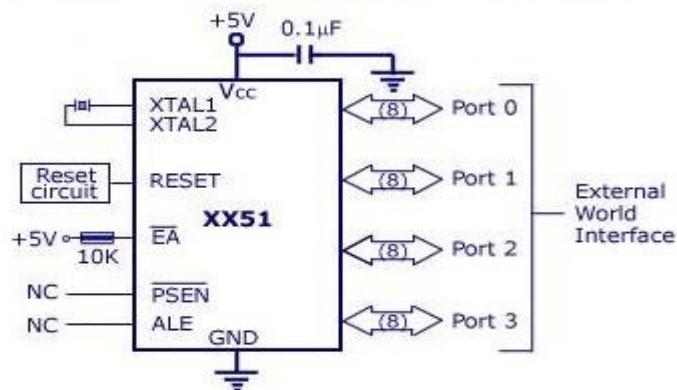


The general schematic diagram of 8051 microcontroller is shown above. We can see 3 system inputs, 3 control signals and 4 ports (for external interfacing). A V_{CC} power supply and ground is also shown. Now let's explain and go through each in detail. System inputs are necessary to make the micro controller functional. So the first and most important of this is power, marked as V_{CC} with a GND (ground potential). Without proper power supply, no electronic system would work. XTAL 1 and XTAL 2 are for the system clock inputs from crystal clock circuit. RESET input is required to initialize microcontroller to default/desired values and to make a new start.

There are 3 control signals, EA, PSEN and ALE. These signals known as External Access (EA), Program Store Enable (PSEN), and Address Latch Enable (ALE) are used for external memory interfacing.

Take a look at the **schematic diagram below** (a functional microcontroller)

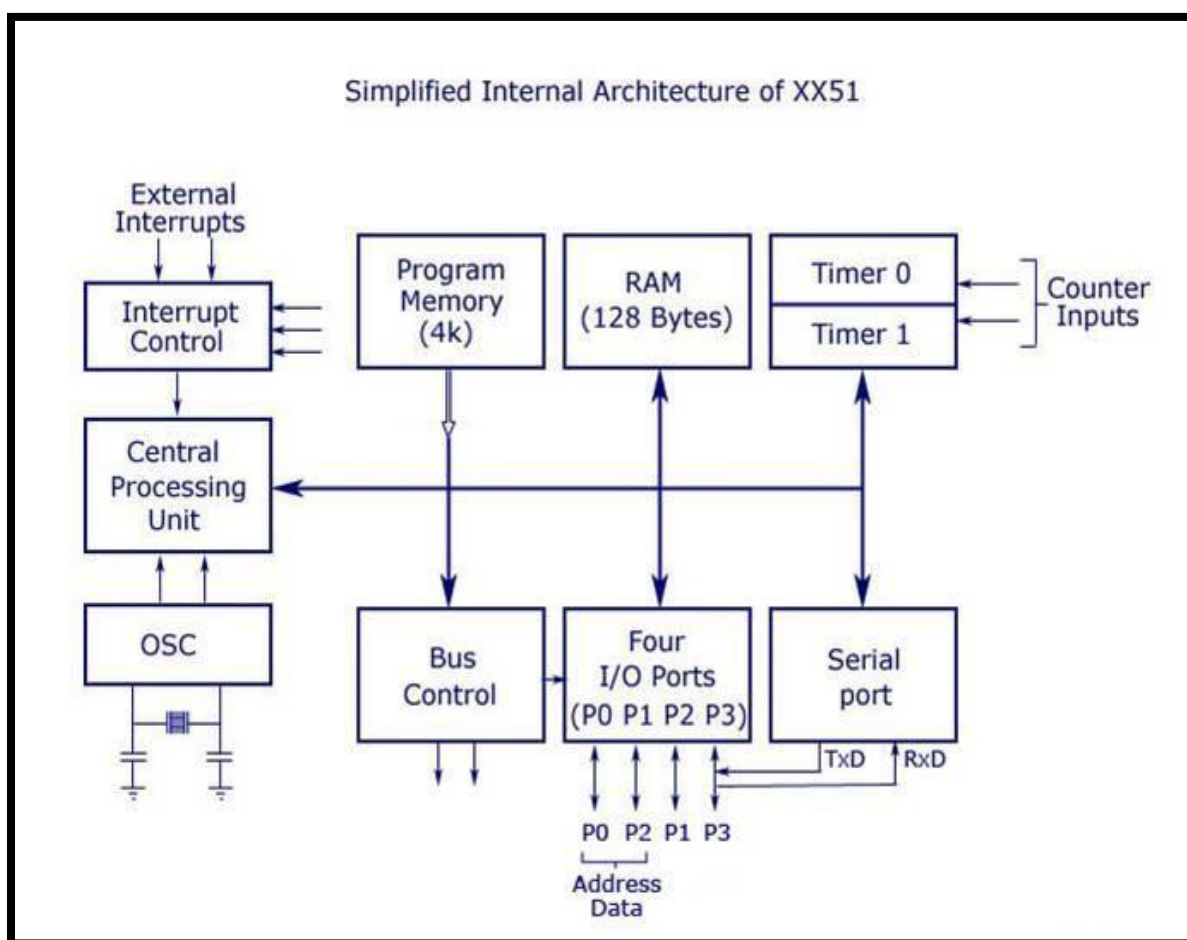
Schematic System Inputs for Stand Alone Operation



As mentioned above, control signals are used for external memory interfacing. If there is no requirement of external memory interfacing then, EA pin is pulled high (connected to Vcc) and two others PSEN and ALE are left alone. You can also see a 0.1 micro farad decoupling capacitor connected to Vcc (to avoid HF oscillations at input).

There are four ports numbered 0,1,2,3 and called as Port 0, Port 1, Port 2 and Port 3 which are used for external interfacing of devices like DAC, ADC, 7 segment display, LED etc. Each port has 8 I/O lines and they all are bit programmable.

2.1 8051 Internal Architecture



There is no need of any detailed explanation to understand internal architecture of 8051 micro controller. Just look at the diagram above and you observe it carefully. The system bus connects all the support devices with the central processing unit. 8051 system bus composes of an 8 bit data bus and a 16 bit address bus and bus control signals. From the figure you can understand that all other devices like program memory, ports, data memory, serial interface,

interrupt control, timers, and the central processing unit are all interfaced together through the system bus. RxD and TxD (serial port input and output) are interfaced with port 3.

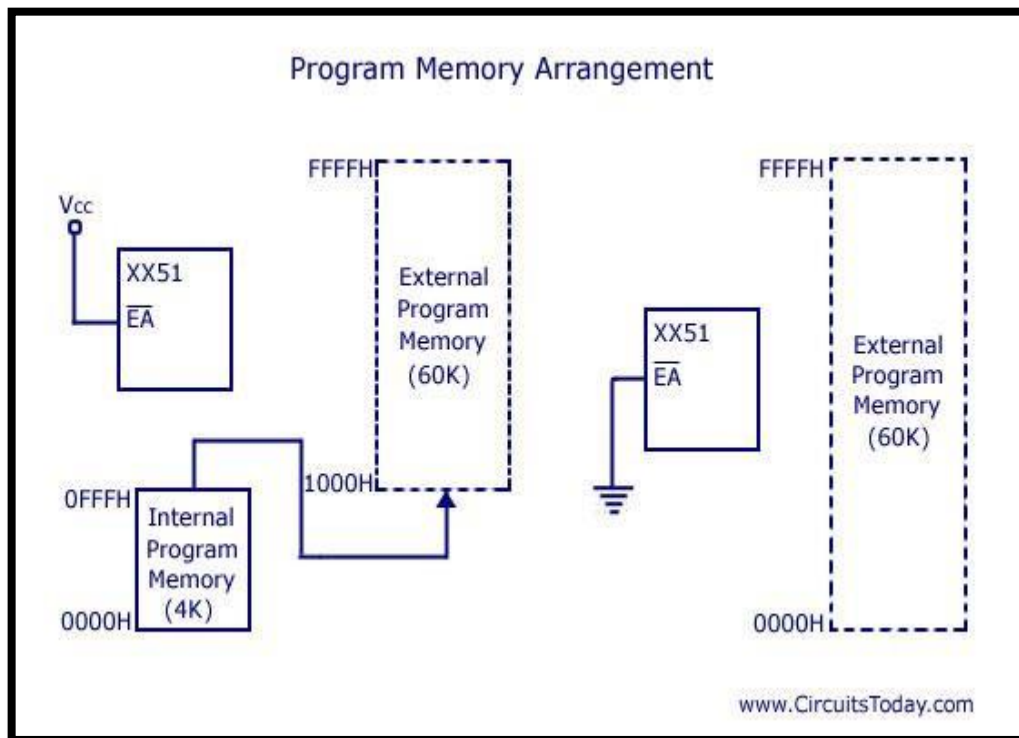
3. 8051 MEMORY ORGANIZATION

Before going deep into the memory architecture of 8051, let's talk a little bit about two variations available for the same. They are Princeton architecture and Harvard architecture. Princeton architecture treats address memory and data memory as a single unit (does not distinguish between two) whereas Harvard architecture treats program memory and data memory as separate entities. Thus Harvard architecture demands address, data and control bus for accessing them separately whereas Princeton architecture does not demand any such separate bus.

Example: 8051 micro controller is based on Harvard architecture and 8085 micro processor is based on Princeton architecture.

Thus 8051 have two memories: *Program memory and Data memory*

- **Program memory organization**

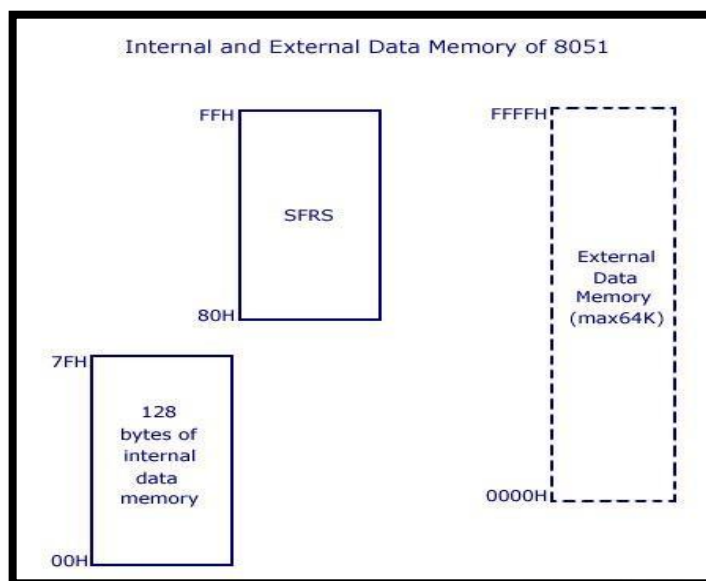


Now let's dive into the program memory organization of 8051. It has an internal program of 4K size and if needed an external memory can be added (by interfacing) of size 60K maximum. So in total 64K size memory is available for 8051 micro controller. By default, the External Access (EA) pin should be connected Vcc so that instructions are fetched from internal memory initially. When the limit of internal memory (4K) is crossed, control will automatically move to external memory to fetch remaining instructions. If the programmer wants to fetch instruction from external memory only (bypassing the internal memory), then he must connect External Access (EA) pin to ground (GND).

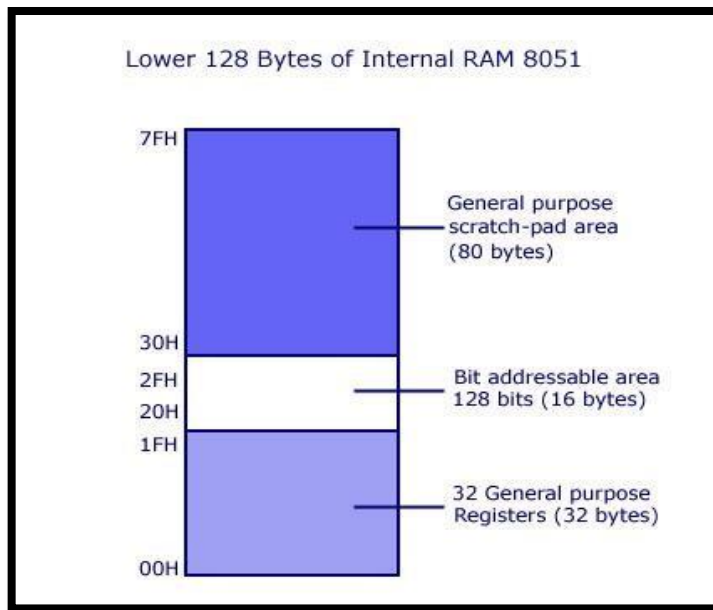
You may already know that 8051 has a special feature of locking the program memory (internal) and hence protecting against software piracy. This feature is enabled by program lock bits. Once these bits are programmed, contents of internal memory can not be accessed using an external circuitry. However locking the software is not possible if external memory is also used to store the software code. Only internal memory can be locked and protected. Once locked, these bits can be unlocked only by a memory-erase operation, which in turn will erase the programs in internal memory too.

8051 is capable of pipelining. Pipelining makes a processor capable of fetching the next instruction while executing previous instruction. It's something like multi tasking, doing more than one operation at a time. 8051 is capable of fetching first byte of the next instruction while executing the previous instruction.

- **Data memory organization**



In the MCS-51 family, 8051 has 128 bytes of internal data memory and it allows interfacing external data memory of maximum size up to 64K. So the total size of data memory in 8051 can be upto 64K (external) + 128 bytes (internal). Observe the diagram carefully to get more understanding. So there are 3 separations/divisions of the data memory:- 1) Register banks 2) Bit addressable area 3) Scratch pad area.

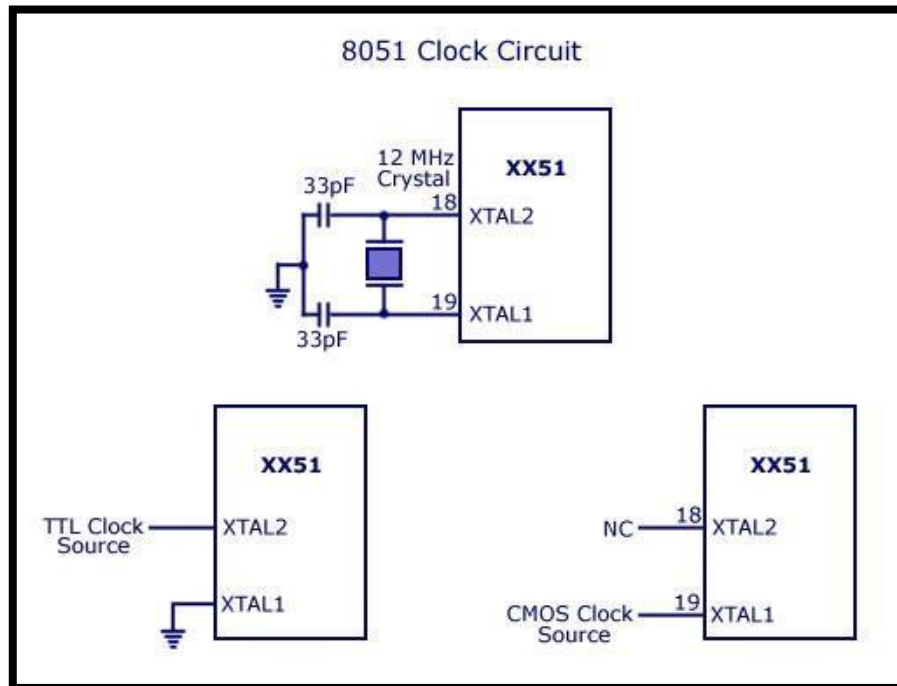


Register banks form the lowest 32 bytes on internal memory and there are 4 register banks designated bank #0, #1, #2 and #3. Each bank has 8 registers which are designated as R0, R1...R7. At a time only one register bank is selected for operations and the registers inside the selected bank are accessed using mnemonics R0..R1.. etc. Other registers can be accessed simultaneously only by direct addressing. Registers are used to store data or operands during executions. By default register bank #0 is selected (after a system reset).

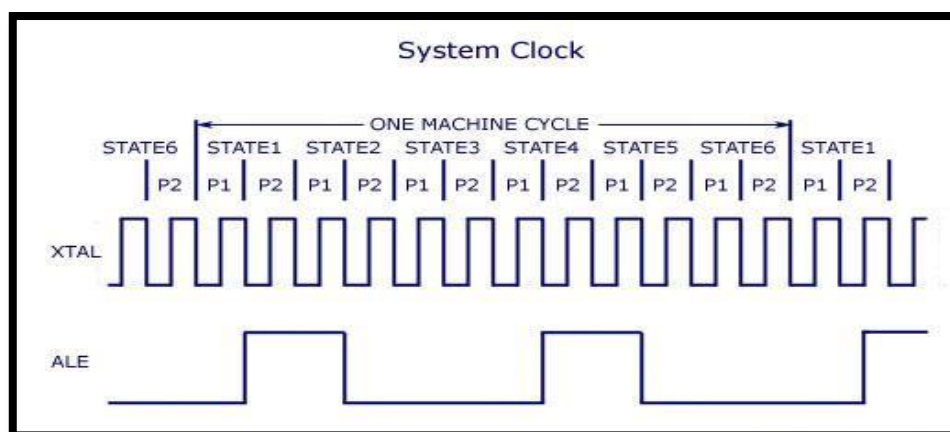
The bit addressable area of 8051 is usually used to store bit variables. The bit addressable area is formed by the 16 bytes next to register banks. They are designated from address 20H to 2FH (total 128 bits). Each bit can be accessed from 00H to 7FH within this 128 bits from 20H to 2FH. Bit addressable area is mainly used to store bit variables from application program, like status of an output device like LED or Motor (ON/OFF) etc. We need only a bit to store this status and using a complete byte addressable area for storing this is really bad programming practice, since it results in wastage of memory.

The scratch pad area is the upper 80 bytes which is used for general purpose storage. Scratch pad area is from 30H to 7FH and this includes stack too.

8051 System Clock

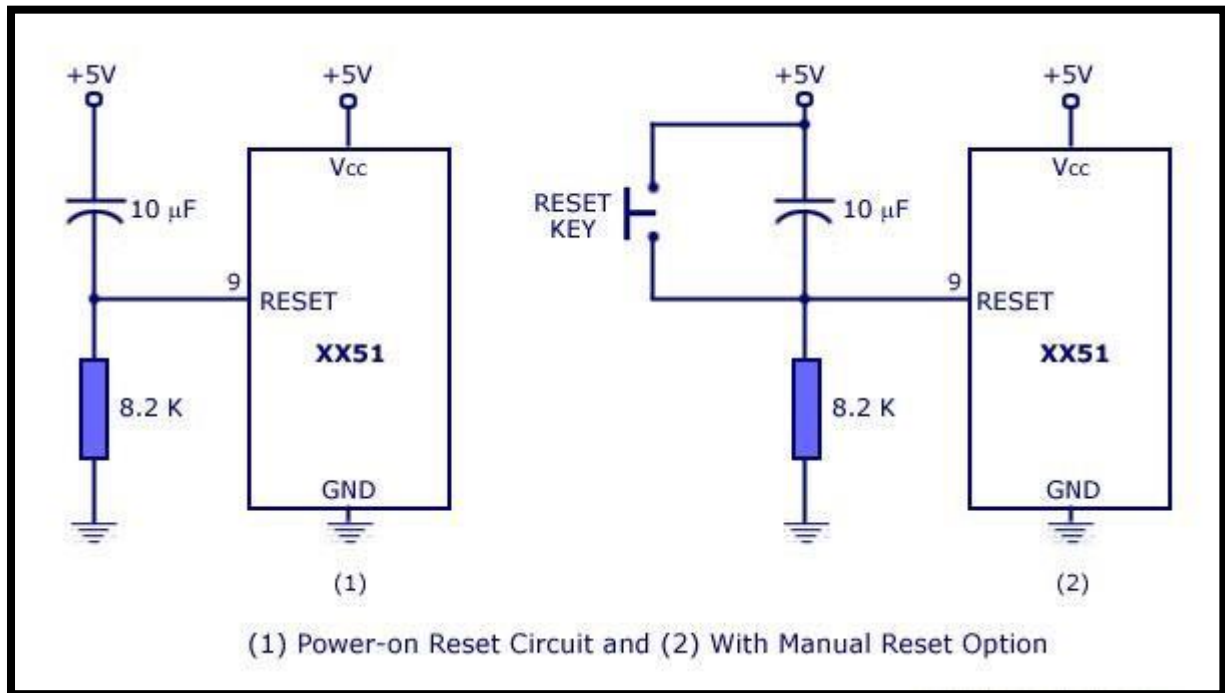


In general cases, a quartz crystal is used to make the clock circuit. The connections are shown in figure (a) and note the connections to XTAL 1 and XTAL 2. In some cases external clock sources are used and you can see the various connections above. Clock frequency limits (maximum and minimum) may change from device to device. Standard practice is to use 12MHz frequency. If serial communications are involved then its best to use 11.0592 MHz frequency.



Okay, take a look at the above machine cycle waveform. One complete oscillation of the clock source is called a pulse. Two pulses forms a state and six states forms one machine cycle. Also note that, two pulses of ALE are available for 1 machine cycle.

8051 Reset Circuit



8051 can be reset in two ways 1) is power-on reset – which resets the 8051 when power is turned ON and 2) manual reset – in which a reset happens only when a push button is pressed manually. Two different reset circuits are shown above. A reset doesn't affect contents of internal RAM. For reset to happen, the reset input pin (pin 9) must be active high for atleast 2 machine cycles. During a reset operation :- Program counter is cleared and it starts from 00H, register bank #0 is selected as default, Stack pointer is initialized to 07H, all ports are written with FFH.

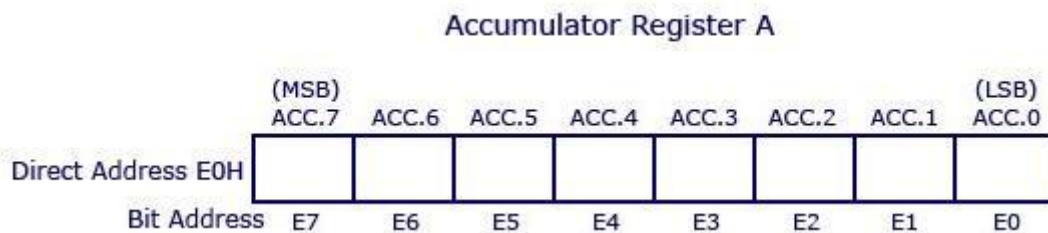
4. 8051 SPECIAL FUNCTION REGISTERS AND PORT OPERATIONS

There are **21 Special function registers (SFR)** in 8051 micro controller and this includes Register A, Register B, Processor Status Word (PSW), PCON etc etc. There are 21 unique locations for these 21 special function registers and each of these register is of 1 byte size.

Some of these special function registers **are bit addressable** (which means you can access 8 individual bits inside a single byte), while some others are **only byte addressable**. Let's take a look at them in detail.

Register A/Accumulator

The most important of all special function registers, that's the first comment about **Accumulator** which is also known as **ACC** or **A**. The Accumulator (sometimes referred to as Register A also) holds the result of most of arithmetic and logic operations. ACC is usually accessed by direct addressing and its physical address is **E0H**. Accumulator is **both byte and bit addressable**. You can understand this from the figure shown below. To access the first bit (i.e bit 0) or to access accumulator as a single byte (all 8 bits at once), you may use the same physical address E0H. Now if you want to access the second bit (i.e., bit 1), you may use E1H and for third bit E2H and so on.



Register B

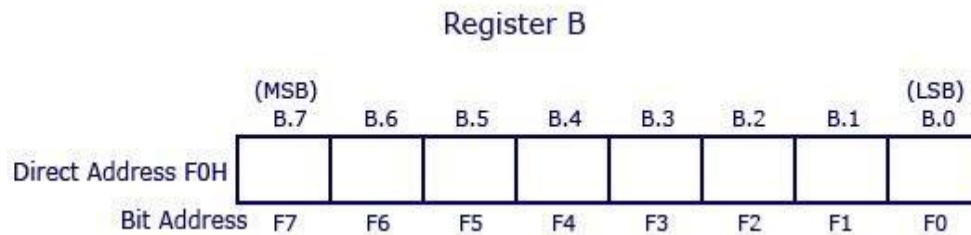
The major purpose of this register is in executing multiplication and division. The 8051 micro controller has a single instruction for multiplication (**MUL**) and division (**DIV**). If you are familiar with 8085, you may now know that multiplication is repeated addition, where as division is repeated subtraction. While programming 8085, you may have written a loop to execute repeated addition/subtraction to perform multiplication and division. Now here in 8051 you can do this with a single instruction.

Ex: MUL A,B – When this instruction is executed, data inside A and data inside B is multiplied and answer is stored in A.

Note: MUL and DIV instructions, it is necessary that the two operands must be in A and B.

Note: Follow this link if you are interested in knowing about differences between a microprocessor and microcontroller.

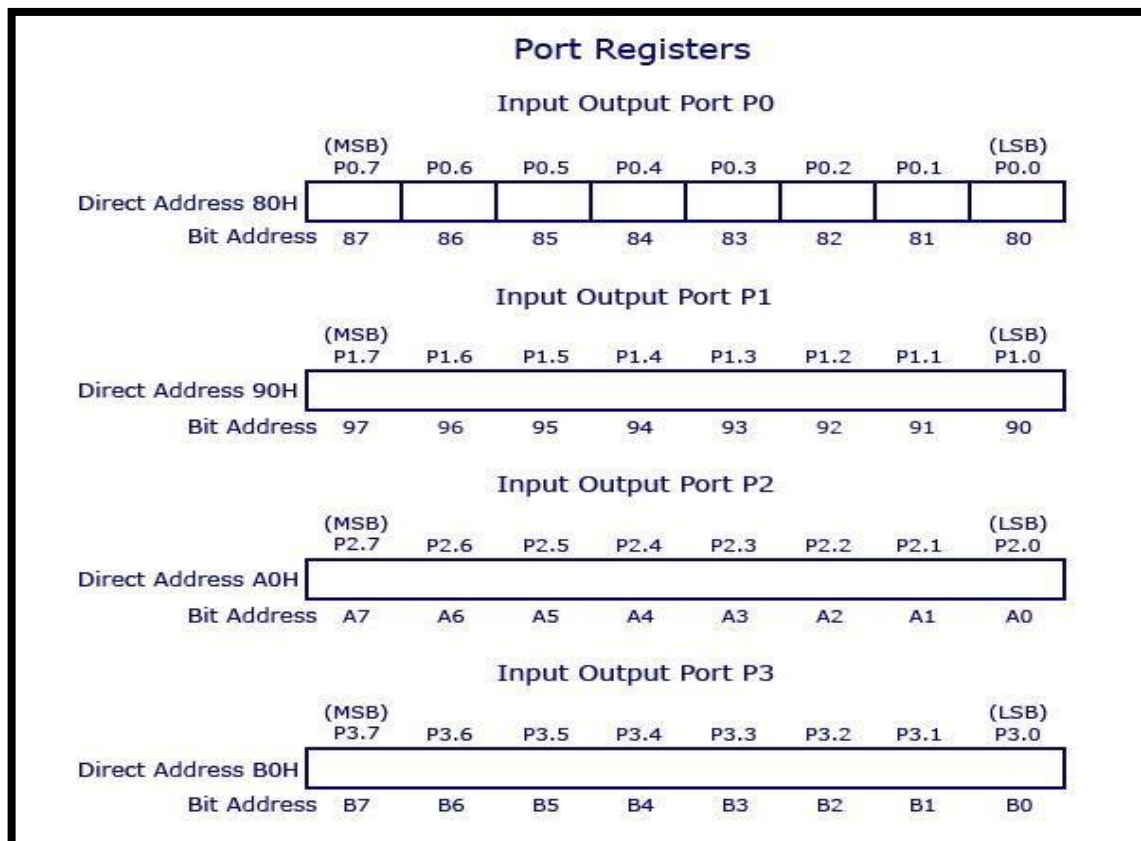
Register B is also byte addressable and bit addressable. To access bit 0 or to access all 8 bits (as a single byte), physical address F0 is used. To access bit 1 you may use F1 and so on. Please take a look at the picture below.



Note: Register B can also be used for other general purpose operations.

PORT REGISTERS

As you may already know, there are 4 ports for 8051. So 4 Input/Output ports named P0, P1, P2 and P3 has got four corresponding port registers with same name P0, P1, P2 and P3. Data must be written into port registers first to send it out to any other external device through ports. Similarly any data received through ports must be read from port registers for performing any operation. All 4 port registers are bit as well as byte addressable. Take a look at the figure below for a better understanding of port registers.



From the figure:

- The physical address of port 0 is 80
- The physical address of port 1 is 90
- And that of port 2 is A0
- And that of port 3 is B0

Stack Pointer

Known popularly with an acronym SP, stack pointer represents a pointer to the the system stack. Stack pointer is an 8 bit register, the direct address of SP is 81H and it is only byte addressable, which means you can't access individual bits of stack pointer. The content of the stack pointer points to the last stored location of system stack. To store something new in system stack, the SP must be incremented by 1 first and then execute the "store" command. Usually after a system reset SP is initialized as 07H and data can be stored to stack from 08H onwards. This is usually a default case and programmer can alter values of SP to suit his needs.

Power Management Register (PCON)

Power management using a microcontroller is something you see every day in mobile phones. Haven't you noticed and got wondered by a mobile phone automatically going into standby mode when not used for a couple of seconds or minutes? This is achieved by power management feature of the controller used inside that phone.

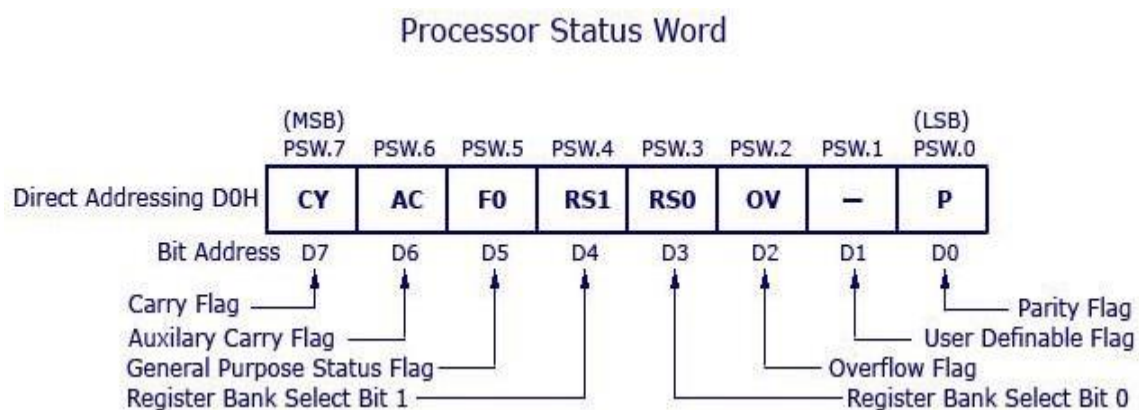
As the name indicates, this register is used for efficient power management of 8051 micro controller. Commonly referred to as PCON register, this is a dedicated SFR for power management alone. From the figure below you can observe that there are 2 modes for this register: **Idle mode and Power down mode**.



Setting bit 0 will move the micro controller to Idle mode and Setting bit 1 will move the micro controller to Power down mode.

Processor Status Word (PSW)

Commonly known as the PSW register, this is a vital SFR in the functioning of micro controller. This register reflects the status of the operation that is being carried out in the processor. The picture below shows PSW register and the way register banks are selected using PSW register bits – RS1 and RS0. PSW register is both bit and byte addressable. The physical address of PSW starts from D0H. The individual bits are then accessed using D1, D2 ... D7. The various individual bits are explained below.

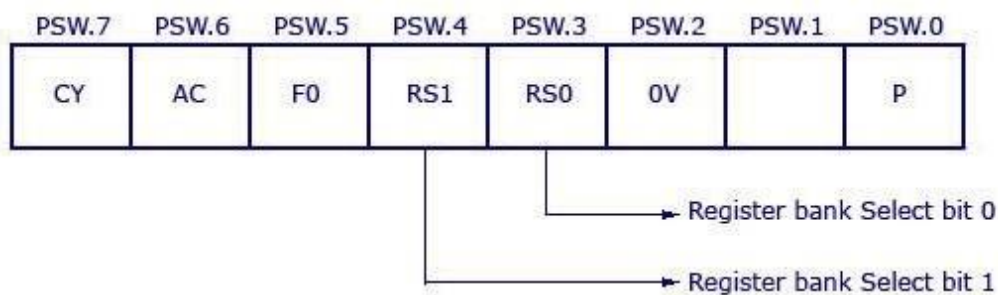


Bit No	Bit Symbol	Direct Address	Name	Function
0	P	D0	Parity	This bit will be set if ACC has odd number of 1's after an operation. If not, bit will remain cleared.
1	-	D1		User definable bit
2	OV	D2	Overflow	OV flag is set if there is a carry from bit 6 but not from bit 7 of an Arithmetic operation. It's also set if there is a carry from bit 7 (but not from bit 6) of Acc
3	RS0	D3	Register Bank select	LSB of the register bank select bit. Look for explanation below this table.

			bit 0	
4	RS1	D4	Register Bank select bit 1	MSB of the register bank select bits.
5	F0	D5	Flag 0	User defined flag
6	AC	D6	Auxiliary carry	This bit is set if data is coming out from bit 3 to bit 4 of Acc during an Arithmetic operation.
7	CY	D7	Carry	Is set if data is coming out of bit 7 of Acc during an Arithmetic operation.

At a time registers can take value from R0,R1...to R7. You may already know there are 32 such registers. There are 4 register banks named 0,1,2 and 3. Each bank has 8 registers named from R0 to R7. At a time only one register bank can be selected. Selection of register bank is made possible through PSW register bits PSW.3 and PSW.4, named as RS0 and RS1. These two bits are known as register bank select bits as they are used to select register banks. The picture will talk more about selecting register banks.

Processor Status Word



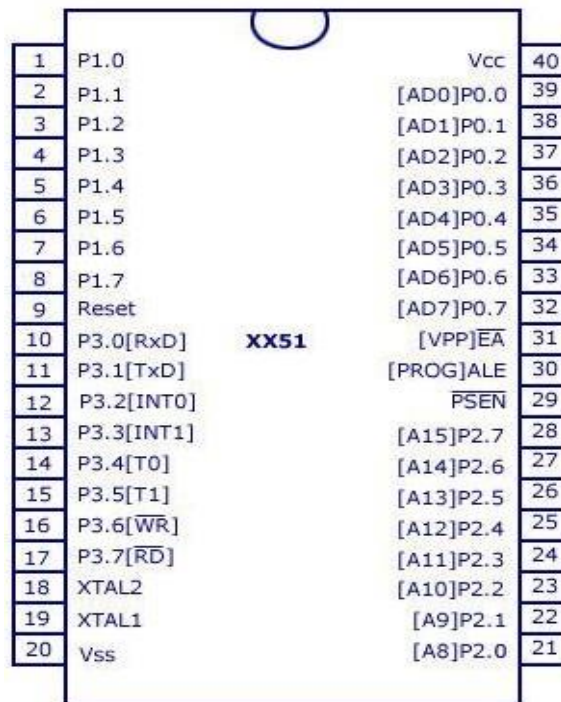
RS1	RS0	Register Bank	Register Bank Status
0	0	0	Register Bank 0 is selected
0	1	1	Register Bank 1 is selected
1	0	2	Register Bank 2 is selected
1	1	3	Register Bank 3 is selected

So far we have discussed about all major SFR's in 8051. There many other still waiting! Please remember there are 21 SFR's and we have discussed only 9 specifically. The table below lists all other 12 SFR's.

SFR	Address	Function
DPH	83	Data pointer registers (High). Only byte addressing possible.
DPL	82	Data pointer register (Low). Only byte addressing possible.
IP	B8	Interrupt priority. Both bit addressing and byte addressing possible.
IE	A8	Interrupt enable. Both bit addressing and byte addressing possible.
SBUF	99	Serial Input/Output buffer. Only byte addressing is possible.
SCON	98	Serial communication control. Both bit addressing and byte addressing possible.
TCON	88	Timer control. Both bit addressing and byte addressing possible.
TH0	8C	Timer 0 counter (High). Only byte addressing is possible.
TL0	8A	Timer 0 counter (Low). Only byte addressing is possible.
TH1	8D	Timer 1 counter (High). Only byte addressing is possible.
TL1	8B	Timer 1 counter (Low). Only byte addressing is possible.
TMOD	89	Timer mode select. Only byte addressing is possible.

5. 8051 PINS AND SIGNALS

Pin-Wise Signal Assignment of XX51



For describing pin diagram and pin configuration of 8051, we are taking into consideration a 40 pin DIP (Dual inline package). Now let's go through pin configuration in detail.

- **Pin-40 :** Named as Vcc is the main power source. Usually its +5V DC. You may note some pins are designated with two signals (shown in brackets).
- **Pins 32-39:** Known as Port 0 (P0.0 to P0.7) – In addition to serving as I/O port, lower order address and data bus signals are multiplexed with this port (to serve the purpose of external memory interfacing). This is a bi directional I/O port (the only one in 8051) and external pull up resistors are required to function this port as I/O.
- **Pin-31:-** ALE aka Address Latch Enable is used to demultiplex the address-data signal of port 0 (for external memory interfacing.) 2 ALE pulses are available for each machine cycle.
- **Pin-30:-** EA/ External Access input is used to enable or disallow external memory interfacing. If there is no external memory requirement, this pin is pulled high by connecting it to Vcc.
- **Pin- 29:-** PSEN or Program Store Enable is used to read signal from external program memory.

- **Pins- 21-28:-** Known as Port 2 (P 2.0 to P 2.7) – in addition to serving as I/O port, higher order address bus signals are multiplexed with this quasi bi directional port.
- **Pin 20:-** Named as Vss – it represents ground (0 V) connection.
- **Pins 18 and 19:-** Used for interfacing an external crystal to provide system clock.
- **Pins 10 – 17:-** Known as Port 3. This port also serves some other functions like interrupts, timer input, control signals for external memory interfacing RD and WR , serial communication signals RxD and TxD etc. This is a quasi bi directional port with internal pull up.
- **Pin 9:-** As explained before RESET pin is used to set the 8051 microcontroller to its initial values, while the microcontroller is working or at the initial start of application. The RESET pin must be set high for 2 machine cycles.
- **Pins 1 – 8:-** Known as Port 1. Unlike other ports, this port does not serve any other functions. Port 1 is an internally pulled up, quasi bi directional I/O port.

6. 8051 TIMERS AND COUNTERS

The 8051 has two timers: timer0 and timer1. They can be used either as timers or as counters. Both timers are 16 bits wide. Since the 8051 has an 8-bit architecture, each 16-bit is accessed as two separate registers of low byte and high byte. First we shall discuss about Timer0 registers.

Timer0 registers is a 16 bits register and accessed as low byte and high byte. The low byte is referred as a TL0 and the high byte is referred as TH0. These registers can be accessed like any other registers.

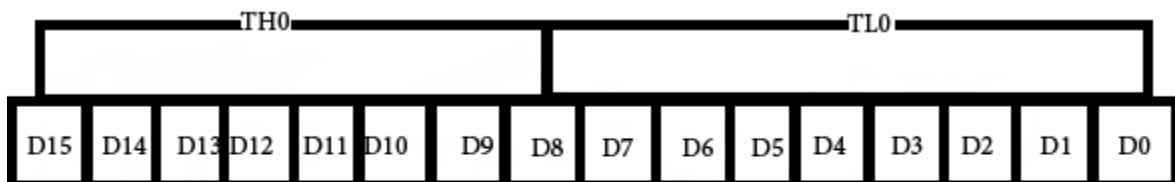


Fig. Timer0

Timer1 registers is also a 16 bits register and is split into two bytes, referred to as TH1 and TL1.

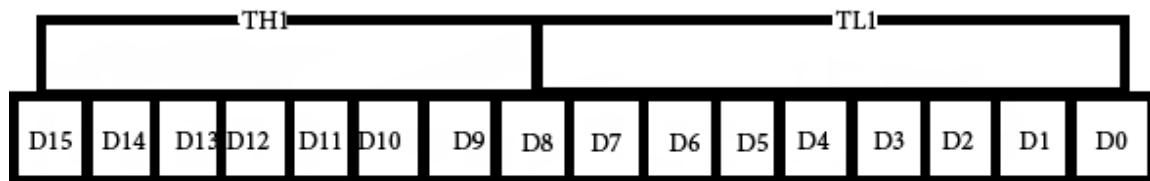


Fig. Timer1

TMOD (timer mode) Register: This is an 8-bit register which is used by both timers 0 and 1 to set the various timer modes. In this TMOD register, lower 4 bits are set aside for timer0 and the upper 4 bits are set aside for timer1. In each case, the lower 2 bits are used to set the timer mode and upper 2 bits to specify the operation.

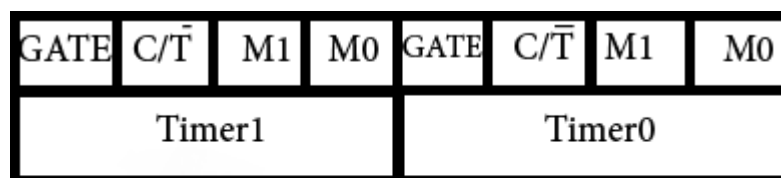


Fig. TMOD Register

TMOD

In upper or lower 4 bits, first bit is a GATE bit. Every timer has a means of starting and stopping. Some timers do this by software, some by hardware, and some have both software and hardware controls. The hardware way of starting and stopping the timer by an external source is achieved by making GATE=1 in the TMOD register. And if we change to GATE=0 then we do not need external hardware to start and stop the timers.

The second bit is C/T bit and is used to decide whether a timer is used as a time delay generator or an event counter. If this bit is 0 then it is used as a timer and if it is 1 then it is used as a counter.

In upper or lower 4 bits, the last bits third and fourth are known as M1 and M0 respectively. These are used to select the timer mode.

M0	M1	M2	Operating Mode
0	0	0	13-bit timer mode, 8-bit timer/counter THx and TLx as 5-bit prescaler.
0	1	1	16-bit timer mode, 16-bit timer/counters THx and TLx are cascaded; There are no prescaler.
1	0	2	8-bit auto reload mode, 8-bit auto reload timer/counter; THx holds a value which is to be reloaded into TLx each time it overflows.
1	1	3	Spilt timer mode.

Mode 1- It is a 16-bit timer; therefore it allows values from 0000 to FFFFH to be loaded into the timer's registers TL and TH. After TH and TL are loaded with a 16-bit initial value, the timer must be started. We can do it by "SETB TR0" for timer 0 and "SETB TR1" for timer 1. After the timer is started. It starts count up until it reaches its limit of FFFFH. When it rolls over from FFFF to 0000H, it sets high a flag bit called TF (timer flag). This timer flag can be monitored. When this timer flag is raised, one option would be stop the timer with the instructions "CLR TR0" or CLR TR1 for timer 0 and timer 1 respectively. Again, it must be noted that each timer flag TF0 for timer 0 and TF1 for timer1. After the timer reaches its limit and rolls over, in order to repeat the process the registers TH and TL must be reloaded with the original value and TF must be reset to 0.

Mode0- Mode 0 is exactly same like mode 1 except that it is a 13-bit timer instead of 16-bit. The 13-bit counter can hold values between 0000 to 1FFFH in TH-TL. Therefore, when the timer reaches its maximum of 1FFFH, it rolls over to 0000, and TF is raised.

Mode 2- It is an 8 bit timer that allows only values of 00 to FFH to be loaded into the timer's register TH. After TH is loaded with 8 bit value, the 8051 gives a copy of it to TL. Then the timer must be started. It is done by the instruction "SETB TR0" for timer 0 and "SETB TR1" for timer1. This is like mode 1. After timer is started, it starts to count up by incrementing the TL register. It counts up until it reaches its limit of FFH. When it rolls over from FFH to 00. It sets high the TF (timer flag). If we are using timer 0, TF0 goes high; if using TF1 then TF1 is raised. When Tl register rolls from FFH to 00 and TF is set to 1, TL is reloaded automatically with the original value kept by the TH register. To repeat the process, we must simply clear TF and let it go without any need by the programmer to reload the

original value. This makes mode 2 auto reload, in contrast in mode 1 in which programmer has to reload TH and TL.

Mode3- Mode 3 is also known as a split timer mode. Timer 0 and 1 may be programmed to be in mode 0, 1 and 2 independently of similar mode for other timer. This is not true for mode 3; timers do not operate independently if mode 3 is chosen for timer 0. Placing timer 1 in mode 3 causes it to stop counting; the control bit TR1 and the timer 1 flag TF1 are then used by timer0.

TCON register- Bits and symbol and functions of every bits of TCON are as follows:



Fig. TCON Register

BIT	Symbol	Functions
7	TF1	Timer1 over flow flag. Set when timer rolls from all 1s to 0. Cleared when the processor vectors to execute interrupt service routine located at program address 001Bh
6	TR1	Timer 1 run control bit. Set to 1 by programmer to enable timer to count; Cleared to 0 by program to halt timer.
5	TF0	Timer 0 over flow flag. Same as TF1.
4	TR0	Timer 0 run control bit. Same as TR1.
3	IE1	External interrupt 1 Edge flag. Not related to timer operations.
2	IT1	External interrupt1 signal type control bit. Set to 1 by program to Enable external interrupt 1 to be triggered by a falling edge signal. Set to 0 by program to enable a low level signal on external interrupt1 to generate an interrupt.
1	IE0	External interrupt 0 Edge flag. Not related to timer operations.
0	IT0	External interrupt 0 signal type control bit. Same as IT0.

7. 8051 ADDRESSING MODES

MOV A,#6AH

Here the data 6A is the operand, often known as source data. When this instruction is executed, the data 6AH is moved to accumulator A. There are 5 different ways to execute this instruction and hence we say, we have got 5 addressing modes for 8051. They are **1) Immediate addressing mode** **2) Direct addressing mode** **3) Register direct addressing mode** **4) Register indirect addressing mode** **5) Indexed addressing mode**.

1. Immediate Addressing Mode

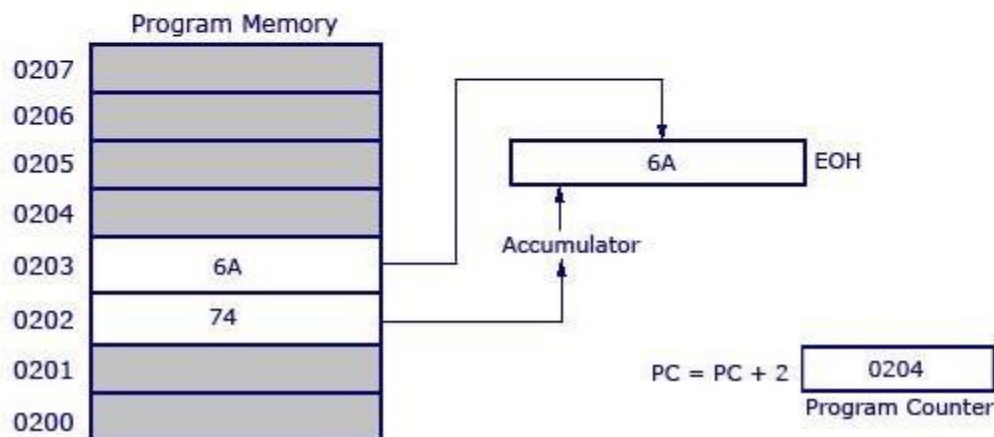
MOV A, #6AH

In general we can write MOV A, #data

This addressing mode is named as “*immediate*” because it transfers an 8-bit data immediately to the accumulator (destination operand).

Immediate Addressing Mode

Instruction	Opcode	Bytes	Cycles
MOV A, #6AH	74H	2	1



The picture above describes the above instruction and its execution. The opcode for MOV A, # data is 74H. The opcode is saved in program memory at 0202 address. The data 6AH is saved in program memory 0203. (See, any part of the program memory can be used, this is just an example) When the opcode 74H is read, the next step taken would be to transfer

whatever data at the next program memory address (here at 0203) to accumulator A (E0H is the address of accumulator). This instruction is of two bytes and is executed in one cycle. So after the execution of this instruction, program counter will add 2 and move to 0204 of program memory.

Note: The '#' symbol before 6AH indicates that operand is a data (8 bit). If '#' is not present then the hexadecimal number would be taken as address.

2. Direct Addressing Mode

This is another way of addressing an operand. Here the address of the data (source data) is given as operand. Let's take an example.

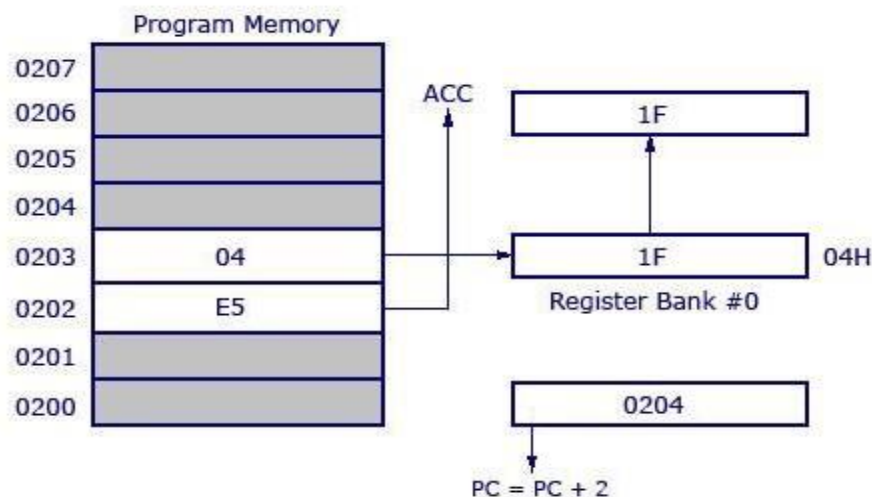
MOV A, 04H

Here 04H is the address of register 4 of register bank#0. When this instruction is executed, whatever data is stored in register 04H is moved to accumulator. In the picture below we can see, register 04H holds the data 1FH. So the data 1FH is moved to accumulator.

Note: We have not used '#' in direct addressing mode, unlike immediate mode. If we had used '#', the data value 04H would have been transferred to accumulator instead of 1FH.

Direct Addressing Mode

Instruction	Opcode	Bytes	Cycles
MOV A, #04H	E5	2	1



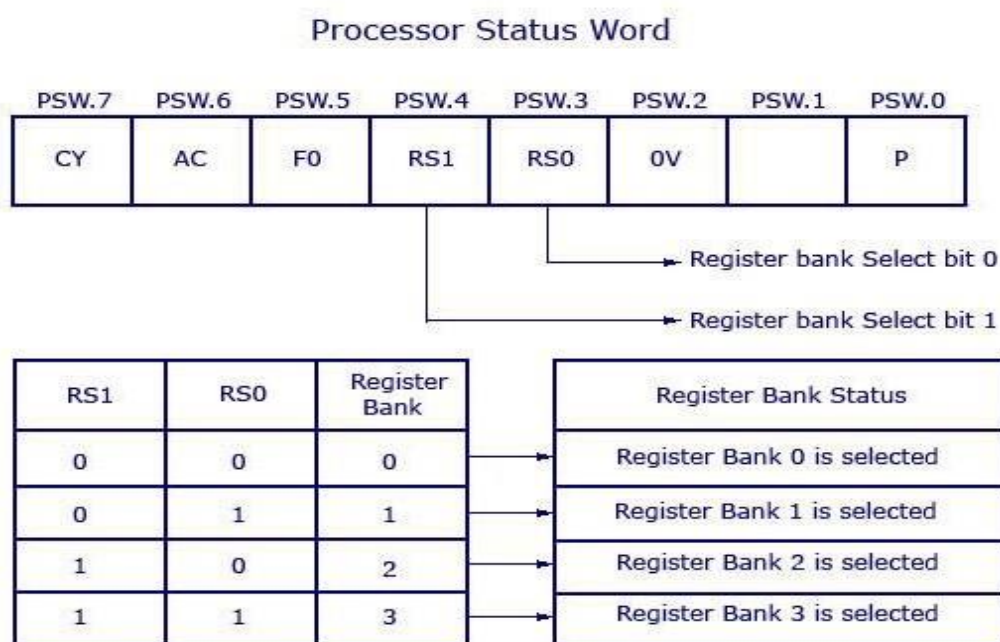
As shown in picture above this is a 2 byte instruction which requires 1 cycle to complete. Program counter will increment by 2 and stand in 0204. The opcode for instruction **MOV A, address** is E5H. When the instruction at 0202 is executed (E5H), accumulator is made active and ready to receive data. Then program control goes to next address that is 0203 and look up the address of the location (04H) where the source data (to be transferred to accumulator) is located. At 04H the control finds the data 1F and transfers it to accumulator and hence the execution is completed.

3. Register Direct Addressing Mode

In this addressing mode we use the register name directly (as source operand). An example is shown below.

MOV A, R4

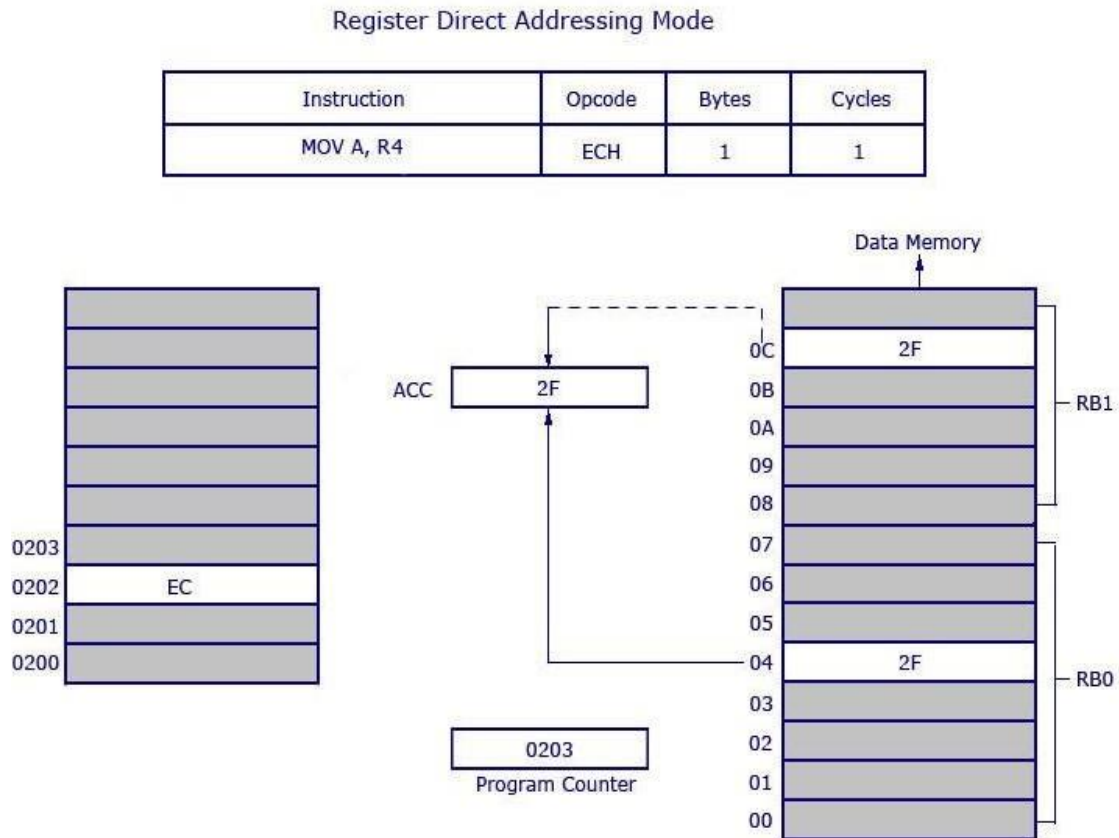
At a time registers can take value from R0, R1...to R7. You may already know there are 32 such registers. There are 4 register banks named 0, 1, 2 and 3. Each bank has 8 registers named from R0 to R7. At a time only one register bank can be selected. Selection of register bank is made possible through a Special Function Register (SFR) named Processor Status Word (PSW). PSW is an 8 bit SFR where each bit can be programmed. Bits are designated from PSW.0 to PSW.7. Register banks are selected using PSW.3 and PSW.4. These two bits are known as register bank select bits as they are used to select register banks. A picture below shows the PSW register and the Register Bank Select bits with status.



IT-T44 MICROPROCESSORS AND MICROCONTROLLERS

So in register direct addressing mode, data is transferred to accumulator from the register (based on which register bank is selected).

Take a look at the picture below.



So we see that opcode for MOV A, R4 is EC. The opcode is stored in program memory address 0202 and when it is executed the control goes directly to R4 of the respected register bank (that is selected in PSW). If register bank #0 is selected then the data from R4 of register bank #0 will be moved to accumulator. (Here it is 2F stored at 04 H). 04 H is the address of R4 of register bank #0. Movement of data (2F) in this case is shown as bold line. Now please take a look at the dotted line. Here 2F is getting transferred to accumulator from data memory location 0C H. Now understand that 0C H is the address location of Register 4 (R4) of register bank #1. Programmers usually get confused with register bank selection. Also keep in mind that data at R4 of register bank #0 and register bank #1 (or even other banks) will not be same. So wrong selection of register banks will result in undesired output. Also note that the instruction above is 1 byte and requires 1 cycle for complete execution. This means using register direct addressing mode can save program memory.

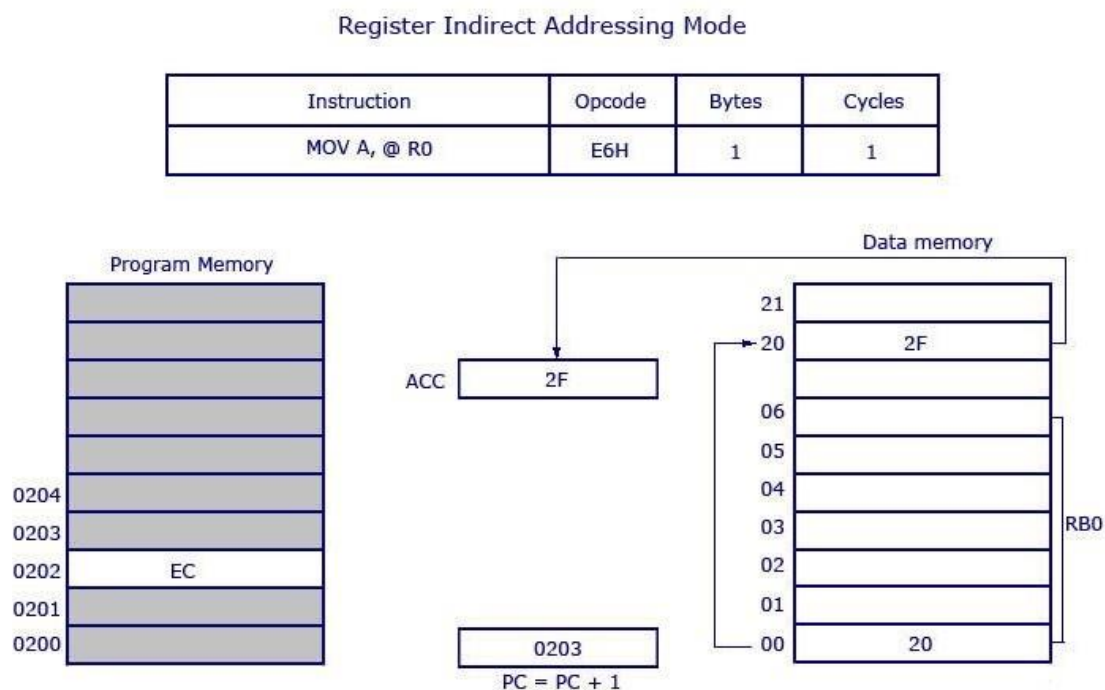
4. Register Indirect Addressing Mode

So in this addressing mode, address of the data (source data to transfer) is given in the register operand.

MOV A, @R0

Here the value inside R0 is considered as an address, which holds the data to be transferred to accumulator.

Example: If R0 holds the value 20H, and we have a data 2F H stored at the address 20H, then the value 2FH will get transferred to accumulator after executing this instruction. **Got it?** See the picture given below,



So the opcode for **MOV A, @R0** is E6H. Assuming that register bank #0 is selected. So the R0 of register bank #0 holds the data 20H. Program control moves to 20H where it locates the data 2FH and it transfers 2FH to accumulator.

This is a single byte instruction and the program counter increments 1 and moves to 0203 of program memory.

Note: Only R0 and R1 are allowed to form a register indirect addressing instruction. In other words programmer must make any instruction either using @R0 or @R1. All register banks are allowed.

5. Indexed Addressing Mode

Well lets see two examples first.

MOVC A, @A+DPTR and MOVC A, @A+PC

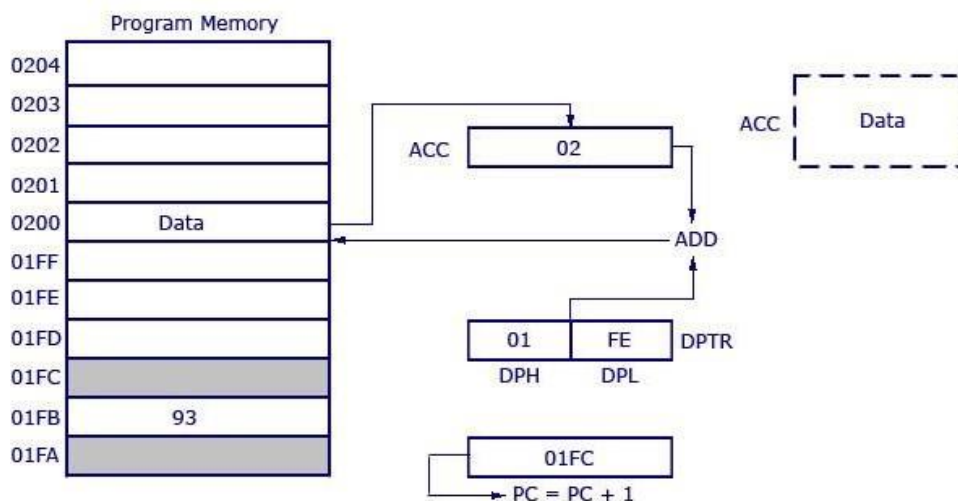
Where DPTR is data pointer and PC is program counter (both are 16 bit registers). Lets take the first example.

MOVC A, @A+DPTR

The source operand is @A+DPTR and we know we will get the source data (to transfer) from this location. It is nothing but adding contents of DPTR with present content of accumulator. This addition will result a new data which is taken as the address of source data (to transfer). The data at this address is then transferred to accumulator. Take a look at the picture below.

Indexed Addressing Mode

Instruction	Opcode	Bytes	Cycles
MOVC A, @A + DPTR	93H	1	2



The opcode for the instruction is 93H. DPTR holds the value 01FE, where 01 is located in DPH (higher 8 bits) and FE is located in DPL (lower 8 bits). Accumulator now has the value 02H. A 16 bit addition is performed and now 01FE H+02 H results in 0200 H. Whatever data is in 0200 H will get transferred to accumulator. The previous value inside accumulator (02H) will get replaced with new data from 0200H. ***New data in the accumulator is shown in dotted line box.***

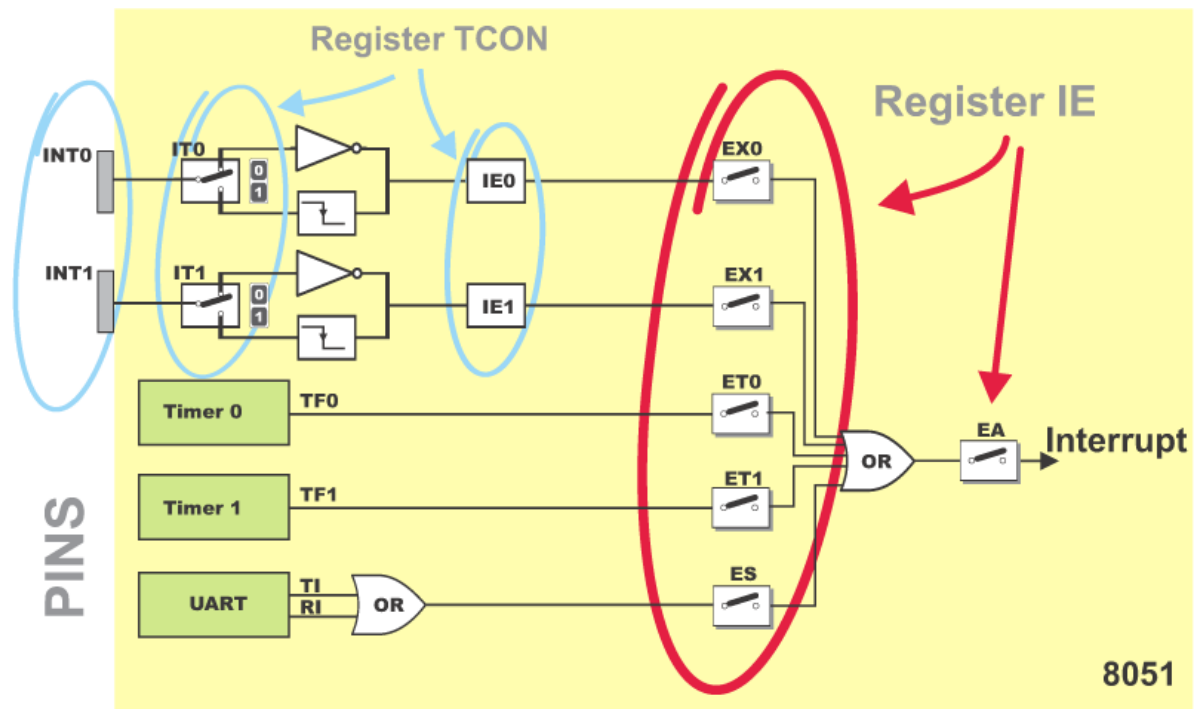
This is a 1 byte instruction **with 2 cycles needed for execution**. What you infer from that? The execution time required for this instruction is high compared to previous instructions (which all were 1 cycle).

The other example **MOVC A, @A+PC** works the same way as above example. The only difference is, instead of adding DPTR with accumulator, here data inside program counter (PC) is added with accumulator to obtain the target address.

8. 8051 MICROCONTROLLER INTERRUPTS

There are five interrupt sources for the 8051, which means that they can recognize 5 different events that can interrupt regular program execution. Each interrupt can be enabled or disabled by setting bits of the IE register. Likewise, the whole interrupt system can be disabled by clearing the EA bit of the same register. Refer to figure below.

Now, it is necessary to explain a few details referring to external interrupts- INT0 and INT1. If the IT0 and IT1 bits of the TCON register are set, an interrupt will be generated on high to low transition, i.e. on the falling pulse edge (only in that moment). If these bits are cleared, an interrupt will be continuously executed as far as the pins are held low.



IE Register (InterruptEnable)

	0	X	0	0	0	0	0	0	Value after Reset
IE	EA		ET2	ES	ET1	EX1	ET0	EX0	Bit name
	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	

- **EA** - global interrupt enable/disable:
 - 0 - disables all interrupt requests.
 - 1 - enables all individual interrupt requests.
- **ES** - enables or disables serial interrupt:
 - 0 - UART system cannot generate an interrupt.
 - 1 - UART system enables an interrupt.
- **ET1** - bit enables or disables Timer 1 interrupt:
 - 0 - Timer 1 cannot generate an interrupt.
 - 1 - Timer 1 enables an interrupt.
- **EX1** - bit enables or disables external 1 interrupt:
 - 0 - change of the pin INT0 logic state cannot generate an interrupt.
 - 1 - enables an external interrupt on the pin INT0 state change.
- **ET0** - bit enables or disables timer 0 interrupt:
 - 0 - Timer 0 cannot generate an interrupt.

- 1 - enables timer 0 interrupt.
- **EX0** - bit enables or disables external 0 interrupt:
 - 0 - change of the INT1 pin logic state cannot generate an interrupt.
 - 1 - enables an external interrupt on the pin INT1 state change.

Interrupt Priorities

It is not possible to foresee when an interrupt request will arrive. If several interrupts are enabled, it may happen that while one of them is in progress, another one is requested. In order that the microcontroller knows whether to continue operation or meet a new interrupt request, there is a priority list instructing it what to do.

The priority list offers 3 levels of interrupt priority:

1. Reset! The absolute master. When a reset request arrives, everything is stopped and the microcontroller restarts.
2. Interrupt priority 1 can be disabled by Reset only.
3. Interrupt priority 0 can be disabled by both Reset and interrupt priority 1.

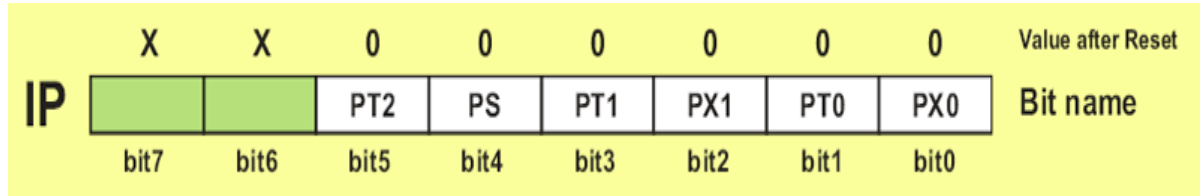
The IP Register (Interrupt Priority Register) specifies which one of existing interrupt sources have higher and which one has lower priority. Interrupt priority is usually specified at the beginning of the program. According to that, there are several possibilities:

- If an interrupt of higher priority arrives while an interrupt is in progress, it will be immediately stopped and the higher priority interrupt will be executed first.
- If two interrupt requests, at different priority levels, arrive at the same time then the higher priority interrupt is serviced first.
- If the both interrupt requests, at the same priority level, occur one after another, the one which came later has to wait until routine being in progress ends.
- If two interrupt requests of equal priority arrive at the same time then the interrupt to be serviced is selected according to the following priority list:
 1. External interrupt INT0
 2. Timer 0 interrupt
 3. External Interrupt INT1
 4. Timer 1 interrupt

5. Serial Communication Interrupt

IP Register (Interrupt Priority)

The IP register bits specify the priority level of each interrupt (high or low priority).



- **PS** - Serial Port Interrupt priority bit
 - Priority 0
 - Priority 1
- **PT1** - Timer 1 interrupt priority
 - Priority 0
 - Priority 1
- **PX1** - External Interrupt INT1 priority
 - Priority 0
 - Priority 1
- **PT0** - Timer 0 Interrupt Priority
 - Priority 0
 - Priority 1
- **PX0** - External Interrupt INT0 Priority
 - Priority 0
 - Priority 1

Handling Interrupt

When an interrupt request arrives the following occurs:

1. Instruction in progress is ended.
2. The address of the next instruction to execute is pushed on the stack.
3. Depending on which interrupt is requested, one of 5 vectors (addresses) is written to the program counter in accordance to the table below:

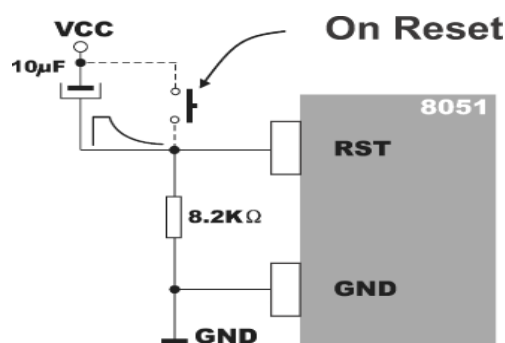
4.

INTERRUPT SOURCE	VECTOR (ADDRESS)
IE0	3 h
TF0	B h
TF1	1B h
RI, TI	23 h
All addresses are in hexadecimal format	

- These addresses store appropriate subroutines processing interrupts. Instead of them, there are usually jump instructions specifying locations on which these subroutines reside.
- When an interrupt routine is executed, the address of the next instruction to execute is popped from the stack to the program counter and interrupted program resumes operation from where it left off.

Reset

Reset occurs when the RS pin is supplied with a positive pulse in duration of at least 2 machine cycles (24 clock cycles of crystal oscillator). After that, the microcontroller generates an internal reset signal which clears all SFRs, except SBUF registers, Stack Pointer and ports (the state of the first two ports is not defined, while FF value is written to the ports configuring all their pins as inputs). Depending on surrounding and purpose of device, the RS pin is usually connected to a power-on reset push button or circuit or to both of them. Figure below illustrates one of the simplest circuits providing safe power-on reset.



Basically, everything is very simple: after turning the power on, electrical capacitor is being charged for several milliseconds through a resistor connected to the ground. The pin is driven high during this process. When the capacitor is charged, power supply voltage is already stable and the pin remains connected to the ground, thus providing normal operation of the microcontroller. Pressing the reset button causes the capacitor to be temporarily discharged and the microcontroller is reset.

9. 8051 INSTRUCTION SET

Introduction

The process of writing program for the microcontroller mainly consists of giving instructions (commands) in the specific order in which they should be executed in order to carry out a specific task. As electronics cannot “understand” what for example an instruction “if the push button is pressed- turn the light on” means, then a certain number of simpler and precisely defined orders that decoder can recognise must be used. All commands are known as INSTRUCTION SET. All microcontrollers compatible with the 8051 have in total of 255 instructions, i.e. 255 different words available for program writing.

At first sight, it is imposing number of odd signs that must be known by heart. However, it is not so complicated as it looks like. Many instructions are considered to be “different”, even though they perform the same operation, so there are only 111 truly different commands. For example: ADD A, R0, ADD A, R1, ... ADD A, R7 are instructions that perform the same operation (addition of the accumulator and register). Since there are 8 such registers, each instruction is counted separately. Taking into account that all instructions perform only 53 operations (addition, subtraction, copy etc.) and most of them are rarely used in practice, there are actually 20-30 abbreviations to be learned, which is acceptable.

Types of instructions

Depending on operation they perform, all instructions are divided in several groups:

- Arithmetic Instructions
- Branch Instructions
- Data Transfer Instructions

- Logic Instructions
- Bit-oriented Instructions

Arithmetic instructions

Arithmetic instructions perform several basic operations such as addition, subtraction, division, multiplication etc. After execution, the result is stored in the first operand. For example:

ADD A,R1 - The result of addition (A+R1) will be stored in the accumulator.

ARITHMETIC INSTRUCTIONS

Mnemonic	Description	Byte	Cycle
ADD A,Rn	Adds the register to the accumulator	1	1
ADD A,direct	Adds the direct byte to the accumulator	2	2
ADD A,@Ri	Adds the indirect RAM to the accumulator	1	2
ADD A,#data	Adds the immediate data to the accumulator	2	2
ADDC A,Rn	Adds the register to the accumulator with a carry flag	1	1
ADDC A,direct	Adds the direct byte to the accumulator with a carry flag	2	2
ADDC A,@Ri	Adds the indirect RAM to the accumulator with a carry flag	1	2
ADDC A,#data	Adds the immediate data to the accumulator with a carry flag	2	2
SUBB A,Rn	Subtracts the register from the accumulator with a borrow	1	1
SUBB A,direct	Subtracts the direct byte from the accumulator with a borrow	2	2
SUBB A,@Ri	Subtracts the indirect RAM from the accumulator with a borrow	1	2
SUBB A,#data	Subtracts the immediate data from the accumulator with a borrow	2	2

INC A	Increments the accumulator by 1	1	1
INC Rn	Increments the register by 1	1	2
INC Rx	Increments the direct byte by 1	2	3
INC @Ri	Increments the indirect RAM by 1	1	3
DEC A	Decrements the accumulator by 1	1	1
DEC Rn	Decrements the register by 1	1	1
DEC Rx	Decrements the direct byte by 1	1	2
DEC @Ri	Decrements the indirect RAM by 1	2	3
INC DPTR	Increments the Data Pointer by 1	1	3
MUL AB	Multiplies A and B	1	5
DIV AB	Divides A by B	1	5
DA A	Decimal adjustment of the accumulator according to BCD code	1	1

Branch Instructions

There are two kinds of branch instructions:

Unconditional jump instructions: upon their execution a jump to a new location from where the program continues execution is executed.

Conditional jump instructions: a jump to a new program location is executed only if a specified condition is met. Otherwise, the program normally proceeds with the next instruction.

BRANCH INSTRUCTIONS

Mnemonic	Description	Byte	Cycle
ACALL addr11	Absolute subroutine call	2	6
LCALL addr16	Long subroutine call	3	6
RET	Returns from subroutine	1	4
RETI	Returns from interrupt subroutine	1	4
AJMP addr11	Absolute jump	2	3
LJMP addr16	Long jump	3	4
SJMP rel	Short jump (from –128 to +127 locations relative to the following instruction)	2	3
JC rel	Jump if carry flag is set. Short jump.	2	3
JNC rel	Jump if carry flag is not set. Short jump.	2	3
JB bit,rel	Jump if direct bit is set. Short jump.	3	4
JBC bit,rel	Jump if direct bit is set and clears bit. Short jump.	3	4
JMP @A+DPTR	Jump indirect relative to the DPTR	1	2
JZ rel	Jump if the accumulator is zero. Short jump.	2	3
JNZ rel	Jump if the accumulator is not zero. Short jump.	2	3
CJNE A,direct,rel	Compares direct byte to the accumulator and jumps if not equal. Short jump.	3	4
CJNE A,#data,rel	Compares immediate data to the accumulator and jumps if not equal. Short jump.	3	4
CJNE Rn,#data,rel	Compares immediate data to the register and jumps if not equal. Short jump.	3	4

CJNE @Ri,#data,rel	Compares immediate data to indirect register and jumps if not equal. Short jump.	3	4
DJNZ Rn,rel	Decrements register and jumps if not 0. Short jump.	2	3
DJNZ Rx,rel	Decrements direct byte and jump if not 0. Short jump.	3	4
NOP	No operation	1	1

Data Transfer Instructions

Data transfer instructions move the content of one register to another. The register the content of which is moved remains unchanged. If they have the suffix “X” (MOVX), the data is exchanged with external memory.

D A T A T R A N S F E R I N S T R U C T I O N S			
Mnemonic	Description	Byte	Cycle
MOV A,Rn	Moves the register to the accumulator	1	1
MOV A,direct	Moves the direct byte to the accumulator	2	2
MOV A,@Ri	Moves the indirect RAM to the accumulator	1	2
MOV A,#data	Moves the immediate data to the accumulator	2	2
MOV Rn,A	Moves the accumulator to the register	1	2
MOV Rn,direct	Moves the direct byte to the register	2	4
MOV Rn,#data	Moves the immediate data to the register	2	2
MOV direct,A	Moves the accumulator to the direct byte	2	3
MOV direct,Rn	Moves the register to the direct byte	2	3
MOV direct,direct	Moves the direct byte to the direct byte	3	4
MOV direct,@Ri	Moves the indirect RAM to the direct byte	2	4

MOV direct,#data	Moves the immediate data to the direct byte	3	3
MOV @Ri,A	Moves the accumulator to the indirect RAM	1	3
MOV @Ri,direct	Moves the direct byte to the indirect RAM	2	5
MOV @Ri,#data	Moves the immediate data to the indirect RAM	2	3
MOV DPTR,#data	Moves a 16-bit data to the data pointer	3	3
MOVC A,@A+DPTR	Moves the code byte relative to the DPTR to the accumulator (address=A+DPTR)	1	3
MOVC A,@A+PC	Moves the code byte relative to the PC to the accumulator (address=A+PC)	1	3
MOVX A,@Ri	Moves the external RAM (8-bit address) to the accumulator	1	3-10
MOVX A,@DPTR	Moves the external RAM (16-bit address) to the accumulator	1	3-10
MOVX @Ri,A	Moves the accumulator to the external RAM (8-bit address)	1	4-11
MOVX @DPTR,A	Moves the accumulator to the external RAM (16-bit address)	1	4-11
PUSH direct	Pushes the direct byte onto the stack	2	4
POP direct	Pops the direct byte from the stack	2	3
XCH A,Rn	Exchanges the register with the accumulator	1	2
XCH A,direct	Exchanges the direct byte with the accumulator	2	3
XCH A,@Ri	Exchanges the indirect RAM with the accumulator	1	3
XCHD A,@Ri	Exchanges the low-order nibble indirect RAM with the accumulator	1	3

Logic Instructions

Logic instructions perform logic operations upon corresponding bits of two registers. After execution, the result is stored in the first operand.

LOGIC INSTRUCTIONS

Mnemonic	Description	Byte	Cycle
ANL A,Rn	AND register to accumulator	1	1
ANL A,direct	AND direct byte to accumulator	2	2
ANL A,@Ri	AND indirect RAM to accumulator	1	2
ANL A,#data	AND immediate data to accumulator	2	2
ANL direct,A	AND accumulator to direct byte	2	3
ANL direct,#data	AND immediate data to direct register	3	4
ORL A,Rn	OR register to accumulator	1	1
ORL A,direct	OR direct byte to accumulator	2	2
ORL A,@Ri	OR indirect RAM to accumulator	1	2
ORL direct,A	OR accumulator to direct byte	2	3
ORL direct,#data	OR immediate data to direct byte	3	4
XRL A,Rn	Exclusive OR register to accumulator	1	1
XRL A,direct	Exclusive OR direct byte to accumulator	2	2
XRL A,@Ri	Exclusive OR indirect RAM to accumulator	1	2
XRL A,#data	Exclusive OR immediate data to accumulator	2	2
XRL direct,A	Exclusive OR accumulator to direct byte	2	3

IT-T44 MICROPROCESSORS AND MICROCONTROLLERS

XORL direct,#data	Exclusive OR immediate data to direct byte	3	4
CLR A	Clears the accumulator	1	1
CPL A	Complements the accumulator (1=0, 0=1)	1	1
SWAP A	Swaps nibbles within the accumulator	1	1
RL A	Rotates bits in the accumulator left	1	1
RLC A	Rotates bits in the accumulator left through carry	1	1
RR A	Rotates bits in the accumulator right	1	1
RRC A	Rotates bits in the accumulator right through carry	1	1

Bit-oriented Instructions

Similar to logic instructions, bit-oriented instructions perform logic operations. The difference is that these are performed upon single bits.

B I T - O R I E N T E D I N S T R U C T I O N S

Mnemonic	Description	Byte	Cycle
CLR C	Clears the carry flag	1	1
CLR bit	Clears the direct bit	2	3
SETB C	Sets the carry flag	1	1
SETB bit	Sets the direct bit	2	3
CPL C	Complements the carry flag	1	1
CPL bit	Complements the direct bit	2	3
ANL C,bit	AND direct bit to the carry flag	2	2

IT-T44 MICROPROCESSORS AND MICROCONTROLLERS

ANL C,/bit	AND complements of direct bit to the carry flag	2	2
ORL C,/bit	OR direct bit to the carry flag	2	2
ORL C,/bit	OR complements of direct bit to the carry flag	2	2
MOV C,/bit	Moves the direct bit to the carry flag	2	2
MOV bit,C	Moves the carry flag to the direct bit	2	3

Description of all 8051 instructions

Here is a list of the operands and their meanings:

- **A**-accumulator;
Rn - is one of working registers (R0-R7) in the currently active RAM memory bank;
- **Direct** - is any 8-bit address register of RAM. It can be any general-purpose register or a SFR (I/O port, control register etc.);
- **@Ri** - is indirect internal or external RAM location addressed by register R0 or R1;
- **#data** - is an 8-bit constant included in instruction (0-255);
- **#data16** - is a 16-bit constant included as bytes 2 and 3 in instruction (0-65535);
- **addr16** - is a 16-bit address. May be anywhere within 64KB of program memory;
- **addr11** - is an 11-bit address. May be within the same 2KB page of program memory as the first byte of the following instruction;
- **rel** - is the address of a close memory location (from -128 to +127 relative to the first byte of the following instruction). On the basis of it, assembler computes the value to add or subtract from the number currently stored in the program counter;
- **bit** - is any bit-addressable I/O pin, control or status bit; and
- **C** - is carry flag of the status register (register PSW).